DESENVOLVIMENTO MOBILE · MÓDULO 06 · TRILHA KOTLIN · NÍVEL INICIANTE

Kotlin Assíncrono: Coroutines e Flow

Thread principal, funções suspensas, dispatchers, concorrência estruturada, Flow e StateFlow

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar por que operações lentas não podem rodar na *thread* principal.
- Escrever funções `suspend` e iniciar coroutines com `launch` e `async`.
- Escolher o *dispatcher* adequado com `withContext`.
- Aplicar concorrência estruturada, cancelamento e tratamento de exceções.
- Produzir e consumir fluxos de dados com `Flow`.
- Expor estado observável com `StateFlow` e eventos com `SharedFlow`.

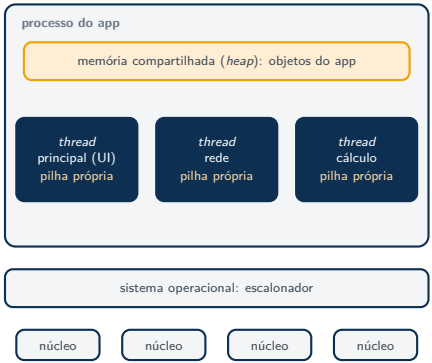
Roteiro

Neste módulo

- Fundamentos: concorrência e assincronia
- Por que programação assíncrona?
- Fundamentos de coroutines
- Flow: fluxos de dados
- StateFlow e SharedFlow
- Fechamento

Fundamentos: concorrência e assincronia

Programas, processos e *threads*



Processo e *thread*

Processo: programa em execução, com memória própria e isolada dos outros apps. ***Thread***: linha de execução dentro do processo; as *threads* dividem a memória do processo, mas cada uma tem sua pilha.

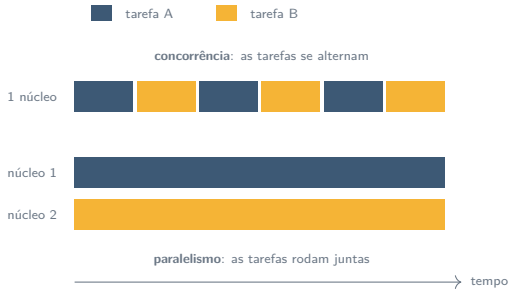
O escalonador

O sistema operacional distribui as *threads* pelos núcleos e alterna entre elas milhares de vezes por segundo (troca de contexto). Cada *thread* custa memória e tempo de troca: não se cria uma por tarefa.

Analogia

O processo é a cozinha; as *threads* são os cozinheiros. Eles dividem a despensa, e é aí que começam as brigas pelo mesmo ingrediente.

Concorrência e paralelismo



Chips de celular

Combinam núcleos de alto desempenho e núcleos econômicos (big.LITTLE): o sistema escolhe onde cada *thread* roda para poupar bateria.

Concorrência

Várias tarefas em andamento no mesmo período, que podem se alternar. É um jeito de **estruturar** o programa.

Paralelismo

Várias tarefas executando **no mesmo instante**, em núcleos diferentes. É um jeito de **executar**.

Na frase de Rob Pike (2012)

Concorrência é lidar com muitas coisas ao mesmo tempo; paralelismo é fazer muitas coisas ao mesmo tempo.

Tarefas de I/O e tarefas de CPU

Tipo	O que limita	Exemplos	Kotlin	Dart
I/O	espera por rede, disco ou banco	requisição HTTP, consulta SQL, ler arquivo	`suspend` + `Dispatchers.IO`	`Future` + `await`
CPU	cálculo no processador	decodificar imagem, JSON gigante, criptografia	`Dispatchers.Default`	`Isolate.run`
Tempo	o relógio	atraso, repetição, tempo limite	`delay`, `withTimeout`	`Future.delayed`, `Timer`

Bloqueante *versus* não bloqueante

Uma chamada **bloqueante** prende a *thread* até terminar. Uma **não bloqueante** registra o pedido e libera a *thread*; o resultado chega depois, por notificação do sistema.

A ideia-chave do módulo

Esperar não precisa de *thread*: enquanto a resposta não chega, a mesma *thread* atende outros eventos. Já o **cálculo pesado precisa de um núcleo** de verdade, fora da *thread* da interface.

O laço de eventos em cada plataforma

Laço de eventos (*event loop*)

Uma *thread* que repete sem parar: pega o próximo evento da fila, executa o código associado até o fim e volta à fila. Toda interface gráfica funciona assim.

Plataforma	Mecanismo na *thread* principal
Android	`Looper` + `MessageQueue` + `Handler`
iOS	`RunLoop` e fila principal do GCD
Dart	fila de microtarefas e fila de eventos
Web	laço de eventos do navegador (tarefas e microtarefas)

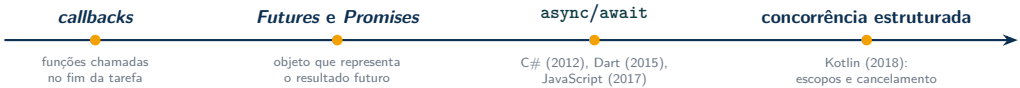
Dart: duas filas

Antes de cada evento, o Dart esvazia a fila de **microtarefas** (continuações curtas de `Futures`). Desde o Flutter 3.29, no Android e no iOS, o código Dart roda na própria *thread* principal do app.

Um evento lento atrasa todos

Quadros se perdem (*jank*) e, no Android, sem resposta a um toque por cerca de 5 s aparece o diálogo ANR.

Da pirâmide de *callbacks* ao `async/await`



Future (ou *Promise*)

Objeto que representa um resultado que ainda não existe. Começa **pendente** e termina **com valor** ou **com erro**. Quem o recebe pode esperar sem bloquear a *thread*.



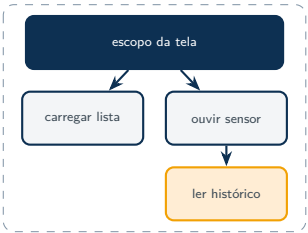
O que acontece em um `await`

A função pausa em um **ponto de suspensão** e devolve a *thread*; quando o resultado chega, continua de onde parou, com as variáveis locais preservadas. O compilador do Kotlin transforma cada função `suspend` em uma máquina de estados que recebe uma `Continuation`.

Dica

O ganho é escrever código assíncrono com cara de sequencial: `try`/`catch` e laços funcionam como sempre.

Concorrência estruturada



a tela fecha ⇒ o escopo termina
⇒ as tarefas filhas são canceladas

Um pouco de história

Assim como a programação estruturada aposentou o `goto`, este modelo aposenta tarefas soltas. A ideia ganhou força em 2018, com um ensaio de Nathaniel J. Smith, e o Kotlin a adotou no mesmo ano.

Concorrência estruturada

Toda tarefa concorrente nasce dentro de um **escopo** e não vive mais que ele: quando o escopo termina, as tarefas filhas são canceladas.

Garantias

Nada fica rodando depois que a tela fecha; o cancelamento chega aos filhos; erros dos filhos chegam ao pai.

E no Dart?

`Future` e `Stream` não seguem esse modelo: cancele inscrições e temporizadores no `dispose` e confira `mounted` após um `await`.

Condições de corrida e impasses



duas somas de 1 a partir de 5 deveriam dar 7: uma atualização se perdeu

Condição de corrida

Erro em que o resultado depende da ordem imprevisível em que tarefas concorrentes acessam um dado compartilhado.

Impasse (*deadlock*)

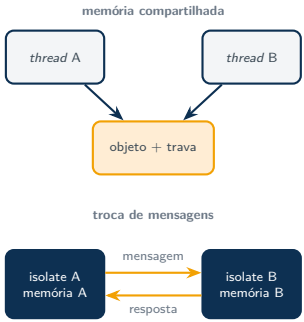
Duas tarefas, cada uma segurando um recurso e esperando o da outra: ninguém avança. Adquira travas sempre na mesma ordem.

Proteção	Como funciona
Exclusão mútua	uma tarefa por vez na região crítica (`Mutex`)
Operação atômica	ler e gravar em um passo indivisível (`AtomicInteger`, `StateFlow.update`)
Confinamento	só uma *thread* toca o dado (a principal, por exemplo)
Imutabilidade	ninguém altera: cria-se uma cópia nova
Mensagens	cada tarefa tem seus dados e troca cópias

Corrida lógica com uma *thread* só

Entre dois `await`, outro evento pode mudar o estado. Duas buscas disparadas em sequência podem responder fora de ordem, e a mais antiga sobrescrever a mais nova.

Memória compartilhada ou troca de mensagens



	Memória compartilhada	Troca de mensagens
Exemplos	*threads* da JVM (Kotlin, Java)	*isolates* do Dart, atores, Web Workers
Comunicação	ler e gravar o mesmo objeto	enviar cópias de dados
Risco	corridas e impasses	custo de copiar dados grandes
Proteção	travas, atômicos, confinamento	o próprio modelo isola

Analogia

Um quadro branco compartilhado exige regras para ninguém apagar o do colega. Colegas que trocam bilhetes nunca brigam pelo quadro, mas gastam tempo escrevendo.

Fluxos de dados: frios, quentes e contrapressão

Fluxo (*stream*)

Sequência de valores que chegam ao longo do tempo: leituras de sensor, mensagens, mudanças no banco. Segue o padrão **Observador**: quem produz notifica quem se inscreveu.

Tipo	Comportamento	Kotlin	Dart
Frio	o produtor começa a cada inscrição	`Flow`	`Stream` (`async*`)
Quente	existe sem observadores; quem chega tarde perde o passado	`SharedFlow`	`Stream` *broadcast*
Estado	quente e guarda o valor atual	`StateFlow`	`ValueNotifier`

Contrapressão (*backpressure*)

O que fazer quando o produtor é mais rápido que o consumidor: acumular (*buffer*), descartar, manter só o mais recente (*conflate*) ou amostrar (*debounce*, *sample*).

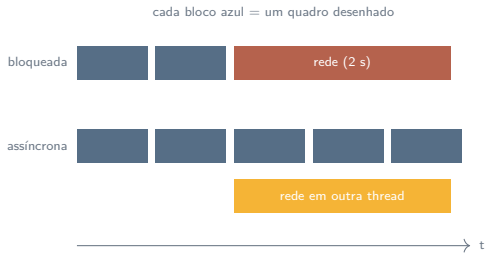
No LabMonitor

Um sensor envia 100 leituras por segundo e a tela desenha 60 quadros: basta mostrar sempre a leitura mais recente.

Por que programação assíncrona?

A *thread* principal (UI thread)

- Todo app tem uma **thread principal** que desenha a tela e responde a toques.
- Para 60 Hz, cada quadro precisa ficar pronto em cerca de **16 ms**.
- Rede, banco de dados, arquivos e processamento pesado levam de dezenas de ms a vários segundos.
- Se a thread principal ficar bloqueada por $\sim$5 s, o sistema mostra o diálogo **ANR** (*Application Not Responding*).



Regra de ouro

Nunca faça I/O nem cálculo pesado na thread principal. Desde o Android 3.0, uma requisição de rede na thread principal gera `NetworkOnMainThreadException`.

Callbacks *versus* coroutines

Callbacks (*callback hell*)

```
login(usuario) { token ->
  buscarBancadas(token) { bancadas ->
    buscarLeituras(bancadas[0]) { leituras ->
      runOnUiThread {
        mostrar(leituras)
      }
    }
  }
}
// erros? cancelamento? cada nível trata o seu
```

Coroutines

```
scope.launch {
  val token = login(usuario)          // suspende
  val bancadas = buscarBancadas(token)
  val leituras = buscarLeituras(bancadas[0])
  mostrar(leituras)                   // de volta à UI
}
// try/catch normal, cancelamento automático
```

Coroutine

Uma computação que pode ser **suspensa** e **retomada** sem bloquear a thread. É muito mais leve que uma thread: dá para ter milhares ao mesmo tempo. O código assíncrono fica com cara de código sequencial.

Fundamentos de coroutines

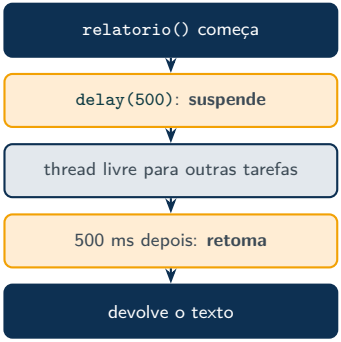
Funções suspend

Suspensão

```
suspend fun lerTemperatura(): Double {
  delay(500)          // suspende, NÃO bloqueia
  return 24.7
}

suspend fun relatorio(): String {
  val t = lerTemperatura()   // pode chamar suspend
  return "Temperatura: $t"
}

fun naoSuspensa() {
  // lerTemperatura()  // ERRO: só dentro de
  //                   // coroutine ou de suspend
}
```



Dica

`suspend` não cria thread nem torna nada assíncrono sozinho: só indica que a função **pode** suspender.

Construtores: `launch`, `async` e `runBlocking`

Iniciando coroutines

```
// launch: "dispare e esqueça", devolve Job
val job: Job = scope.launch {
  salvarLeitura(leitura)
}
job.cancel()

// async: devolve Deferred<T>, use await()
scope.launch {
  val t = async { lerTemperatura() }
  val u = async { lerUmidade() }
  // as duas rodam em paralelo
  mostrar(t.await(), u.await())
}

// runBlocking: BLOQUEIA a thread.
// Só em main() de exemplos e em testes antigos.
fun main() = runBlocking {
  println(relatorio())
}
```

Construtor	Retorno	Uso
`launch`	`Job`	tarefa sem resultado
`async`	`Deferred<T>`	resultado em paralelo
`runBlocking`	`T`	ponte com código bloqueante

Erro comum

Usar `runBlocking` no código do app trava a thread principal, que é justamente o que as coroutines evitam.

Dispatchers e `withContext`

Dispatcher	Quando usar
`Dispatchers.Main`	atualizar a UI; chamadas rápidas
`Dispatchers.IO`	rede, banco, arquivos (I/O bloqueante)
`Dispatchers.Default`	CPU: ordenar, filtrar, FFT, JSON grande

Main-safety

Uma função é *main-safe* quando pode ser chamada da thread principal com segurança, porque ela própria troca de dispatcher. Room, Retrofit e Firebase já são main-safe.

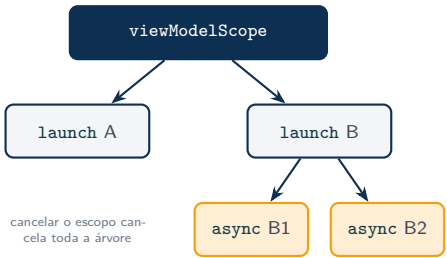
Repositório main-safe

```
class LeituraRepository(
  private val io: CoroutineDispatcher = Dispatchers.IO
) {
  suspend fun exportarCsv(dados: List<Leitura>): File
      =
    withContext(io) {                // troca de thread
      val f = File(dir, "leituras.csv")
      f.writeText(dados.joinToString("\n") {
        "${it.instante};${it.valor}"
      })
      f
    }                                // volta ao
      chamador
}
```

Dica

Injetar o dispatcher permite trocá-lo nos testes (Módulo 14).

Concorrência estruturada



- Toda coroutine nasce dentro de um **CoroutineScope**.
- O pai espera os filhos terminarem.
- Cancelar o pai **cancela todos os filhos**.
- Uma exceção em um filho cancela o pai e os irmãos (exceto com `SupervisorJob`).

Resultado

Nenhuma coroutine fica "solta" consumindo bateria e rede depois que a tela que a criou é fechada.

Escopos no Android

Os três escopos do dia a dia

```kotlin
// 1) No ViewModel: vive enquanto o ViewModel viver
class PainelViewModel : ViewModel() {
  fun atualizar() = viewModelScope.launch {
    _estado.value = repo.carregar()
  }
}

// 2) Em Compose: ligado à composição
@Composable
fun BotaoSincronizar(onSync: suspend () -> Unit) {
  val scope = rememberCoroutineScope()
  Button(onClick = { scope.launch { onSync() } }) {
    Text("Sincronizar")
  }
}

// 3) Em Activity/Fragment: ligado ao ciclo de vida
lifecycleScope.launch { /* ... */ }
```

Escopo	Cancelado quando
`viewModelScope`	o ViewModel é destruído
`rememberCoroutineScope`	o composable sai da tela
`LaunchedEffect`	a chave muda ou sai da tela
`lifecycleScope`	a Activity é destruída

Evite `GlobalScope`

Coroutines em `GlobalScope` não são canceladas automaticamente: causam vazamentos e trabalho desnecessário.

Cancelamento e exceções

Cancelamento é cooperativo

```
val job = scope.launch(Dispatchers.Default) {
  for (amostra in amostras) {
    ensureActive()      // ou: if (!isActive) return
    processar(amostra)  // laço de CPU
  }
}
job.cancel()

// Timeout
val r = withTimeoutOrNull(3_000) {
  conectarSensor()
} ?: "sensor não respondeu"
```

Dica

Funções da biblioteca (`delay`, `withContext`, chamadas Room/Retrofit) já verificam o cancelamento.

Tratando falhas

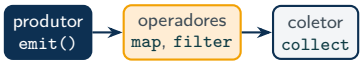
```
viewModelScope.launch {
  try {
    val dados = repo.carregar()
    _estado.value = Estado.Sucesso(dados)
  } catch (e: CancellationException) {
    throw e             // nunca engula!
  } catch (e: IOException) {
    _estado.value = Estado.Erro("Sem conexão")
  }
}

// Filhos independentes: falha de um
// não cancela os outros
val escopo = CoroutineScope(
  SupervisorJob() + Dispatchers.Main
)
```

Flow: fluxos de dados

O que é um `Flow`

- Uma `suspend fun` devolve **um** valor. Um **Flow** emite **vários valores ao longo do tempo**.
- É **frio** (*cold*): o código só roda quando alguém chama `collect`, e roda de novo a cada coletor.
- Ideal para leituras de sensor, atualizações do banco e mensagens em tempo real.



Simulando um sensor

```
fun temperaturaFlow(): Flow<Double> = flow {
  var t = 24.0
  while (true) {
    t += Random.nextDouble(-0.3, 0.3)
    emit(t)           // envia um valor
    delay(1_000)      // 1 Hz
  }
}

// consumindo
scope.launch {
  temperaturaFlow()
    .map { "%.1f °C".format(it) }
    .collect { texto -> println(texto) }
}
```

Operadores úteis

Pipeline reativo

```
val alertas: Flow<String> = temperaturaFlow()
  .onEach { Log.d(TAG, "bruto: $it") }
  .map { it.arredondar(1) }
  .distinctUntilChanged()        // ignora repetidos
  .filter { it > 30.0 }
  .map { "Alerta: $it °C" }
  .catch { e -> emit("Erro: ${e.message}") }
  .flowOn(Dispatchers.Default)   // upstream em Default

// Combinar dois fluxos
val conforto = combine(tempFlow, umidFlow) { t, u ->
  indiceDeCalor(t, u)
}

// Busca com espera de digitação
consultaFlow
  .debounce(300)
  .flatMapLatest { termo -> repo.buscar(termo) }
```

Operador	Efeito
`map`, `filter`	transformar e selecionar
`onEach`	efeito colateral (log)
`distinctUntilChanged`	descarta valores repetidos
`debounce(ms)`	espera o fluxo “acalmar”
`sample(ms)`	um valor a cada período
`combine`	junta os últimos valores de vários fluxos
`flatMapLatest`	troca de fluxo e cancela o anterior
`catch`	trata exceções do upstream
`flowOn`	muda o dispatcher do upstream
`take`, `first`	limita a quantidade

Convertendo callbacks em Flow: `callbackFlow`

Sensor do aparelho como Flow

```
fun SensorManager.leituras(tipo: Int): Flow<FloatArray> =
  callbackFlow {
    val sensor = getDefaultSensor(tipo)
    val listener = object : SensorEventListener {
      override fun onSensorChanged(e: SensorEvent) {
        trySend(e.values.clone())   // envia ao Flow
      }
      override fun onAccuracyChanged(s: Sensor?, a: Int) {}
    }
    registerListener(listener, sensor,
      SensorManager.SENSOR_DELAY_UI)
    // obrigatório: limpa ao cancelar a coleta
    awaitClose { unregisterListener(listener) }
  }
```

Quando usar

APIs baseadas em *listeners*: sensores, localização, BLE, Firebase, MQTT. O `callbackFlow` transforma tudo isso em `Flow`, com cancelamento correto.

Atenção

Esquecer o `awaitClose` deixa o listener registrado, gastando bateria mesmo depois que ninguém mais observa. Voltaremos a esse padrão nos Módulos 12 e 13.

StateFlow e SharedFlow

StateFlow: estado observável

Padrão do ViewModel

```
class PainelViewModel(
  private val repo: LeituraRepository
) : ViewModel() {
  // mutável e privado / somente leitura e público
  private val _ultima = MutableStateFlow<Double?>(null)
  val ultima: StateFlow<Double?> = _ultima.asStateFlow()

  fun simular() {
    _ultima.value = 25.3     // emite para os coletores
    _ultima.update { (it ?: 0.0) + 0.1 }  // atômico
  }

  // Converter um Flow frio em StateFlow
  val media: StateFlow<Double> = repo.leituras()
    .map { lista -> lista.map { it.valor }.average() }
    .stateIn(
      scope = viewModelScope,
      started = SharingStarted.WhileSubscribed(5_000),
      initialValue = 0.0
    )
}
```

StateFlow

Fluxo **quente** que sempre tem um valor atual (`.value`). Novos coletores recebem imediatamente o último valor. Valores iguais em sequência não são reemitidos.

`WhileSubscribed(5_000)`

Mantém o fluxo ativo por 5 s após o último coletor sair. Assim uma rotação de tela não reinicia a consulta, mas sair da tela para tudo.

Na UI (Módulo 07)

```
val v by vm.ultima
  .collectAsStateWithLifecycle()
```

SharedFlow e comparação

Eventos de uso único

```
private val _eventos = MutableSharedFlow<String>()
val eventos = _eventos.asSharedFlow()

fun salvar() = viewModelScope.launch {
  repo.salvar()
  _eventos.emit("Leitura salva!")   // ex.: Snackbar
}
```

Dica

A recomendação atual é modelar até mensagens como parte do estado (ex.: `mensagem: String?` no *UiState*) e limpá-las depois de exibidas. Use `SharedFlow` quando o evento realmente não for estado.

	Flow	**StateFlow**	**SharedFlow**
Tipo	frio	quente	quente
Valor atual	não	sim (`.value`)	não (ou *replay*)
Valor inicial	—	obrigatório	—
Coletores	cada um roda o produtor	compartilham	compartilham
Uso típico	dados do repositório	estado da tela	eventos

Fechamento

Resumo

Resumo

- A thread principal só desenha a UI; I/O e CPU pesada vão para `Dispatchers.IO`/`Default`.
- `suspend` marca funções que podem suspender; `launch` e `async` criam coroutines.
- Use os escopos com ciclo de vida (`viewModelScope`, `rememberCoroutineScope`) e nunca `GlobalScope`.
- Cancelamento é cooperativo; relance `CancellationException`.
- `Flow` representa vários valores no tempo; operadores montam pipelines reativos.
- `StateFlow` expõe o estado da tela; `stateIn` converte fluxos frios.

Glossário (1/2)

Termo	Significado
Thread	linha de execução dentro de um processo, com pilha própria
***Thread* principal**	desenha a interface e trata os toques do usuário
ANR	diálogo exibido quando a *thread* principal deixa de responder
Concorrência	tarefas em andamento no mesmo período, que se alternam
Paralelismo	tarefas executando no mesmo instante, em núcleos diferentes
Coroutine	computação que pode ser suspensa e retomada sem bloquear a *thread*
`suspend`	marca uma função que pode suspender a coroutine
Dispatcher	define em que *threads* a coroutine roda (`Main`, `IO`, `Default`)
Concorrência estruturada	tarefas presas a um escopo: cancelar o escopo cancela as filhas

Glossário (2/2)

Termo	Significado
Cancelamento cooperativo	a coroutine para em pontos de suspensão ou ao verificar `isActive`
`Flow`	fluxo frio de vários valores ao longo do tempo
`StateFlow`	fluxo quente que sempre guarda o valor atual
`SharedFlow`	fluxo quente para eventos, com vários coletores
Condição de corrida	resultado que depende da ordem imprevisível de acesso a um dado
`Mutex`	trava que garante exclusão mútua entre coroutines
Contrapressão	estratégia para quando o produtor é mais rápido que o consumidor

Questões de revisão

1. Por que uma requisição de rede não pode rodar na *thread* principal? O que o usuário veria?
2. Diferencie concorrência de paralelismo com um exemplo no celular.
3. Uma tarefa de I/O e uma de CPU merecem o mesmo tratamento? Que *dispatcher* usar em cada caso?
4. O que acontece com a *thread* quando uma função `suspend` chega a um ponto de suspensão?
5. Explique a concorrência estruturada e o que ela evita quando a tela é fechada.
6. Duas coroutines fazem `contador++` mil vezes cada no `Dispatchers.Default`. Por que o total pode ficar abaixo de 2000? Como corrigir?
7. Qual a diferença entre um `Flow` frio e um `StateFlow`?
8. Um sensor emite 100 leituras por segundo e a tela só precisa da última. Que estratégia de contrapressão usar?

Exercícios

Exercício 1 — Paralelismo

Escreva três funções `suspend` que simulam leituras com `delay` de 1 s, 2 s e 3 s. Meça o tempo total chamando-as em sequência e depois com `async`. Explique a diferença.

Exercício 2 — Flow

Crie um `Flow<Int>` que simula um ADC de 12 bits a 10 Hz. Monte um pipeline que converte para volts, calcula a média de janelas de 10 amostras e emite alerta acima de 3,0 V.

Exercício 3 — Cancelamento

Inicie um laço de CPU com `launch(Dispatchers.Default)` sem `ensureActive()` e cancele após 100 ms. O que acontece? Corrija.

LabMonitor — Etapa 6

Implemente `SimuladorSensor` com um `Flow<Leitura>` por bancada e um `PainelViewModel` que expõe a última leitura e a média dos últimos 60 s via `StateFlow`.

Para aprofundar

- Coroutines no Android: `https://developer.android.com/kotlin/coroutines`
- Boas práticas com coroutines: `https://developer.android.com/kotlin/coroutines/coroutines-best-practices`
- Guia oficial do Kotlin: `https://kotlinlang.org/docs/coroutines-guide.html`
- Flow no Android: `https://developer.android.com/kotlin/flow`
- StateFlow e SharedFlow: `https://developer.android.com/kotlin/flow/stateflow-and-sharedflow`

Respostas das questões de revisão

1. A *thread* principal desenha a tela e trata toques; bloqueá-la congela a interface, perde quadros e, depois de cerca de 5 s, gera um ANR. O Android ainda lança `NetworkOnMainThreadException`.
2. Concorrência: um download e uma animação se alternando em um núcleo. Paralelismo: duas imagens decodificadas ao mesmo tempo em dois núcleos.
3. Não. I/O passa quase todo o tempo esperando (`Dispatchers.IO` ou APIs que já suspendem); CPU ocupa o núcleo (`Dispatchers.Default`).
4. A coroutine pausa e a *thread* fica livre para outras tarefas; quando o resultado chega, ela continua de onde parou, talvez em outra *thread* do mesmo *dispatcher*.
5. Toda coroutine pertence a um escopo e não vive mais que ele. Ao fechar a tela, o escopo (`viewModelScope`, `rememberCoroutineScope`) cancela as filhas: nada de bateria, rede e memória desperdiçadas.
6. `contador++` é ler, somar e gravar: em *threads* diferentes, as duas podem ler o mesmo valor e uma atualização se perde. Use `AtomicInteger`, `Mutex` ou confine o contador a uma *thread*.
7. O `Flow` frio só produz quando coletado e recomeça para cada coletor; o `StateFlow` é quente, sempre tem um valor e entrega o último a quem chega.
8. Manter só o mais recente: `conflate()` ou `collectLatest`. Um `StateFlow` já se comporta assim.

Fim do Módulo 06

Próximo: Módulo 07 — Estado, navegação e animações: remember, state hoisting, efeitos, ciclo de vida, Navigation 3 e animações.